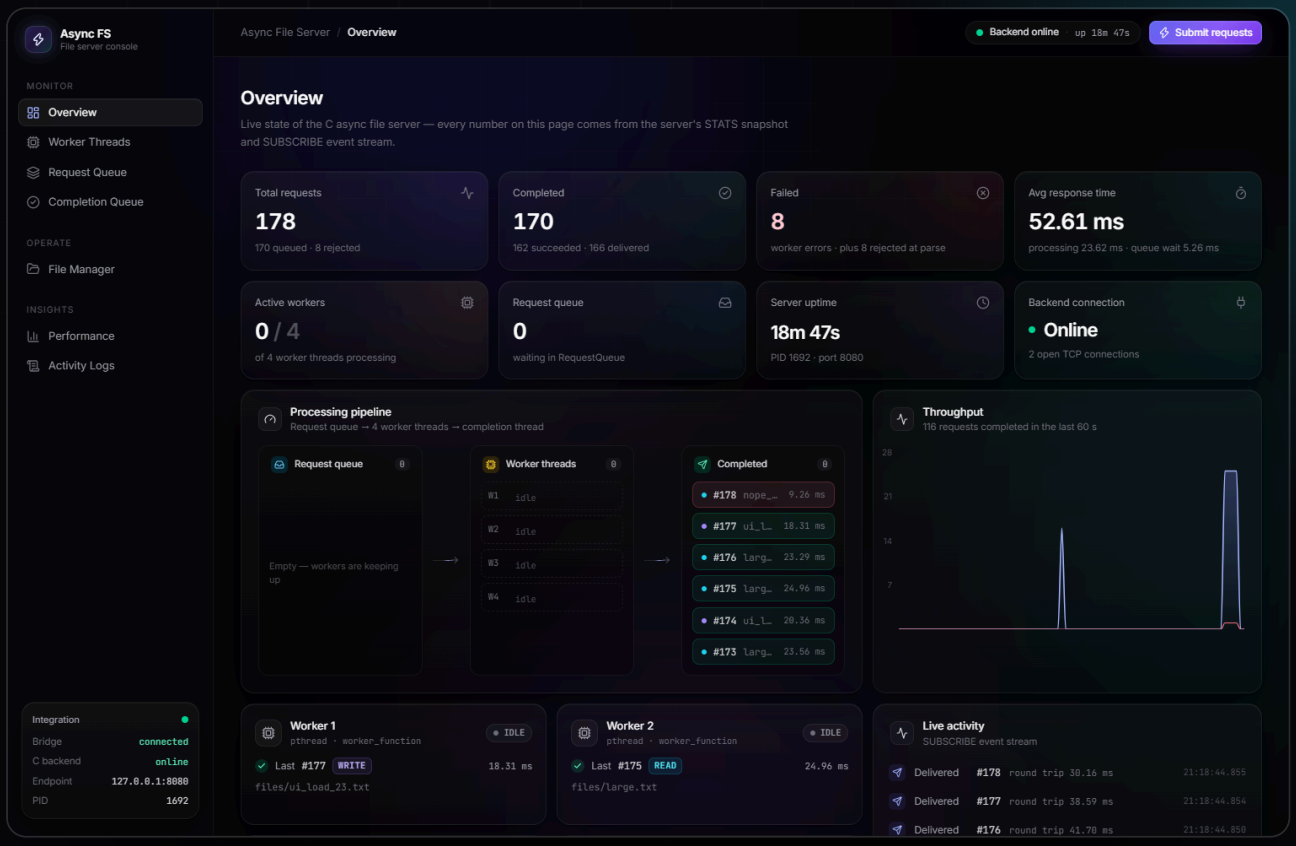
OPERATING SYSTEMS & SYSTEMS PROGRAMMING · PROJECT REPORT

Async File Server Live Console

A multithreaded C file server — request queue, four worker threads and a completion thread — with a real-time web dashboard that controls and monitors it.



ENGINE

C · POSIX threads · TCP

INTEGRATION

Node.js HTTP / WebSocket bridge

DASHBOARD

React · TypeScript · Tailwind

Contents

1	Summary	What the project is and what was built
2	System architecture	Components and how a request flows
3	C backend	Threads, queues, synchronization, file operations
4	Wire protocol	Original and extended commands
5	Correctness and security fixes	Problems found in the original server and how they were fixed
6	Integration bridge	HTTP / WebSocket to TCP
7	Dashboard	The seven pages, with screenshots
8	Testing and results	Protocol, end-to-end, race detection, browser
9	Installation and usage	Exact commands
10	Project structure	Where everything lives

1 Summary

The Async File Server is a C program that accepts READ and WRITE requests over TCP, places them on a shared request queue, processes them concurrently on four POSIX worker threads, and returns results through a separate completion thread. This project turns it into a complete application with a live monitoring and control dashboard.

The C server remains the only processing engine. Every READ, WRITE and CREATE in the dashboard is executed by the C worker threads, and every metric shown — request IDs, queue contents, worker state, timings and logs — is produced by the server itself. A small Node.js bridge translates browser traffic into the server's TCP protocol; it performs no file I/O of its own.

C backend
Thread per client connection, 4 workers, completion thread, reference-counted connections, path-safe file access, live monitor and event stream.

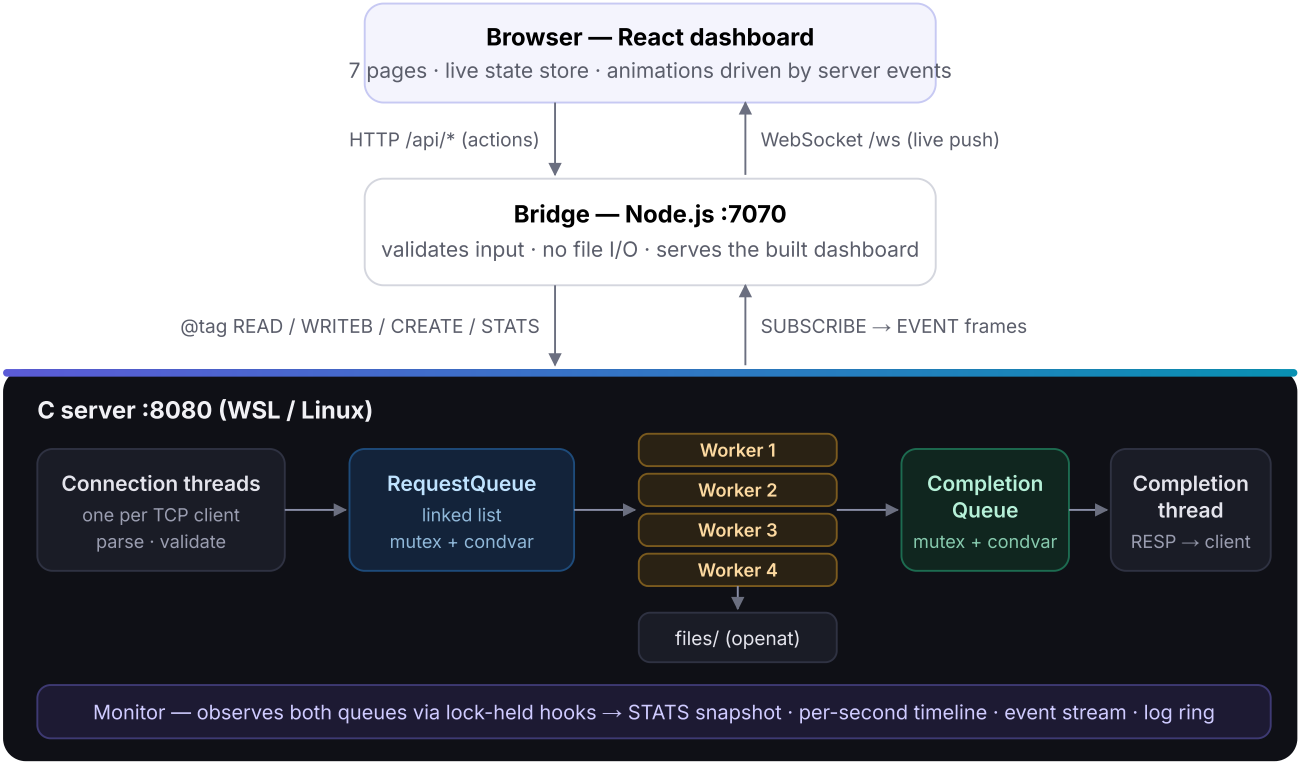
Integration bridge
One pipelined, tagged TCP connection for requests and one subscription connection for live events, exposed as HTTP + WebSocket.

Dashboard
Seven pages: overview, workers, request queue, completion queue, file manager, performance analytics and activity logs.

Verification
65 automated checks across raw TCP and end-to-end suites, ThreadSanitizer clean, browser-tested with zero console errors.

2 System architecture

Three processes cooperate. The browser never talks to the C server directly; the bridge converts between web protocols and the server's line-based TCP protocol.



Lifecycle of one request

STEP	WHAT HAPPENS
1. Submit	The dashboard calls `PUT /api/files/notes.txt`. The bridge validates the name and sends `@b42 WRITEB notes.txt 120\n` followed by 120 bytes on its pipelined connection.
2. Parse	The connection thread reads the line and payload, assigns request ID 57, validates the filename, and calls `enqueue()`. The monitor emits `request_queued`.
3. Dequeue	A worker blocked in `pthread_cond_wait` wakes, takes the request and is marked PROCESSING (`request_started`).
4. Execute	The worker writes to a temporary file in `files/`, renames it over the target, and records CLOCK_MONOTONIC timing with the performance module.
5. Complete	The worker pushes a Completion (`request_finished`) and returns to the queue. The completion thread sends `RESP 57 35 OK b42` and reports delivery (`completion_delivered`).
6. Respond	The bridge matches the tag `b42` to the waiting HTTP request and replies with request ID 57 and status OK. Only now does the dashboard report success.

3 C backend

The backend evolves the original `server_multi_async_workers.c`. Its structure is unchanged — four workers, a completion thread, the same queue and performance APIs and the same log messages — with the concurrency, safety and observability work described below.

Threads

THREAD	COUNT	RESPONSIBILITY
Main	1	Accepts connections, creates a detached connection thread for each, handles SIGINT/SIGTERM and prints the performance summary on shutdown.
Connection	1 per client	Buffered line reader; parses requests; answers monitoring commands directly; enqueues file requests. A `SUBSCRIBE` connection becomes a live event sender.
Worker	4	`dequeue_by()` → validate → READ / WRITE / CREATE → `performance_end()` → `completion_enqueue()`.
Completion	1	`completion_dequeue()` → sends the framed response under the connection's send lock → releases the connection reference.

Synchronization

- **Queues** — linked lists guarded by a mutex and condition variable; consumers sleep in `pthread_cond_wait`.
- **Observer hooks** — the queues call monitor hooks *while holding their own mutex*, so the monitor sees every transition in the exact order it happens and the dashboard reflects the real queue, not a copy.
- **Connections** — reference counted. The reader holds one reference and every queued request holds one, so a socket closes only after its last response.
- **Frames** — a per-connection send lock ensures that error frames and completion frames never interleave on the socket.
- **Lock order** — request queue → completion queue → monitor state → event ring, and logger → event ring. Nothing acquires these in reverse, which rules out deadlock.

File operations

OPERATION	IMPLEMENTATION
`READ`	`openat(files_dir, name, O_RDONLY | O_NOFOLLOW)`, `fstat` to reject non-regular files and files over 16 MB, then a loop of `read()` calls until EOF into a heap buffer.
`WRITE`	Writes all bytes to a temporary file `.afs-tmp—<pid>-<id>`, then `renameat()` over the target. Readers see the old or new content, never a partial file, and concurrent writes cannot interleave.
`CREATE`	`O_CREAT | O_EXCL` — fails with `EEXIST` if the file is already present.

Monitor and logger

The monitor keeps per-worker state, a 500-entry completion history, a 180-second timeline, performance-module aggregates and a 4096-slot event ring. The logger keeps `logs/server.log`, adds levels and request IDs, and holds a 2000-entry ring for `LOGS`.

4 Wire protocol

Requests are newline-terminated text. Every original command behaves exactly as before; the new commands are additions.

REQUEST	ORIGIN	MEANING
`READ <file>`	original	Return the file's contents.
`WRITE <file> <data>`	original	Replace the file with a single line of data.
`EXIT`	original	Close the connection with a goodbye message.
`WRITEB <file> <len>`	new	Followed by exactly *len* raw bytes: multi-line, binary-safe write (up to 8 MB).
`CREATE <file>`	new	Create an empty file; error if it exists.
`LIST · STATS · LOGS [n] · PING`	new	Monitoring; JSON payload, answered immediately by the connection thread.
`SUBSCRIBE`	new	Turn the connection into a live event stream with heartbeats every 2 s.
`@<tag> <request>`	new	Optional tag echoed in the response, so pipelined responses can be matched.

Responses

```
RESP <request_id> <length> <status> [<tag>]\n<payload>
EVENT <seq> <length>\n<json>                    # SUBSCRIBE connections only

# example
@ui7 READ small.txt
RESP 102 31 OK ui7
This is small file for testing
```

`status` is `OK` or an error code: `ENOENT`, `EEXIST`, `EINVAL`, `EACCES`, `ETOOBIG`, `ENOSPC`, `EIO`. The original client parses the header with `sscanf("RESP %d %zu")`, which ignores the added trailing fields — so it continues to work unchanged.

Live events

EVENT	EMITTED WHEN
`request_queued`	A connection thread enqueues a request (includes queue size).
`request_started`	A worker dequeues a request (includes worker ID and queue wait).
`request_finished`	A worker pushes a completion (status, bytes, measured processing time, result preview).
`completion_dispatched / delivered`	The completion thread takes and sends the response (delivered or client gone, end-to-end latency).
`request_rejected`	A request fails validation before reaching the queue.
`client_connected / disconnected · log`	Connection changes and every log line.

5 Correctness and security fixes

Inspecting the original server revealed several defects. Each was fixed with the smallest change that kept the original design and protocol intact.

PROBLEM	EFFECT IN THE ORIGINAL	FIX
Serial clients	The accept loop served one client until it disconnected; a second client (or the dashboard) was blocked.	A detached thread per connection.
fd reuse	A disconnecting client's fd was closed while its requests were still queued. A new client could reuse that fd number and receive another client's response.	Reference-counted connections; the socket closes after the last pending response.
SIGPIPE	Sending to a client that had gone away could terminate the whole server.	`SIGPIPE` ignored, `MSG_NOSIGNAL` on sends.
Interleaved frames	The main thread and completion thread could write to the same socket at once.	Per-connection send lock.
Directory traversal	Filenames such as `../x` were accepted and resolved outside `files/`.	Strict name validation plus `openat(…, O_NOFOLLOW)` on a directory descriptor.
Data race	Four workers updated `performance_stats` without a lock.	Updates are serialized in the monitor.
Short / large reads	A single `read()` call; 1 MB buffers on the worker stack; text-only framing.	Read loop until EOF, heap buffers, length-based framing.
Partial writes	A concurrent READ could observe a half-written file.	Temporary file + atomic `renameat`.
Unmeasured failures	Failed WRITEs skipped the performance module.	`performance_end()` on every path.
Stalled clients	A client that stopped reading could block the completion thread indefinitely.	`SO_SNDTIMEO` on client sockets.
Multi-line data	The line protocol could not carry newlines.	`WRITEB` length-prefixed command.

> **Compatibility.** The original interactive client (`client_multi_async.c`) was compiled unchanged and tested against the new server: single requests, an asynchronous batch and EXIT all work as before.

6 Integration bridge

The bridge is a small Node.js process (one dependency, `ws`). It translates; it does not compute or store file data.

Request connection

One persistent TCP connection. Each HTTP action is sent as `@b<n> <command>` without waiting for earlier responses — the same asynchronous pattern as the CLI batch mode — and resolved only when the server's tagged `RESP` arrives.

Event connection

A `SUBSCRIBE` stream forwarded to every browser over WebSocket. If no heartbeat arrives within 7 s, the backend is marked offline and both connections are re-established.

HTTP API

ENDPOINT	BACKEND COMMAND
`GET /api/status`	Bridge state and backend online/offline with the last error.
`GET /api/stats`	`STATS`
`GET /api/files`	`LIST`
`GET /api/files/:name`	`READ` — returns UTF-8 text, or base64 for binary files.
`PUT /api/files/:name`	`WRITEB`
`POST /api/files`	`CREATE`, then `WRITEB` if initial content is given.
`POST /api/batch`	Up to 64 operations pipelined at once.
`GET /api/logs?limit=`	`LOGS`
`WS /ws`	Status changes, a snapshot every second, and every live event.

Reliability rules

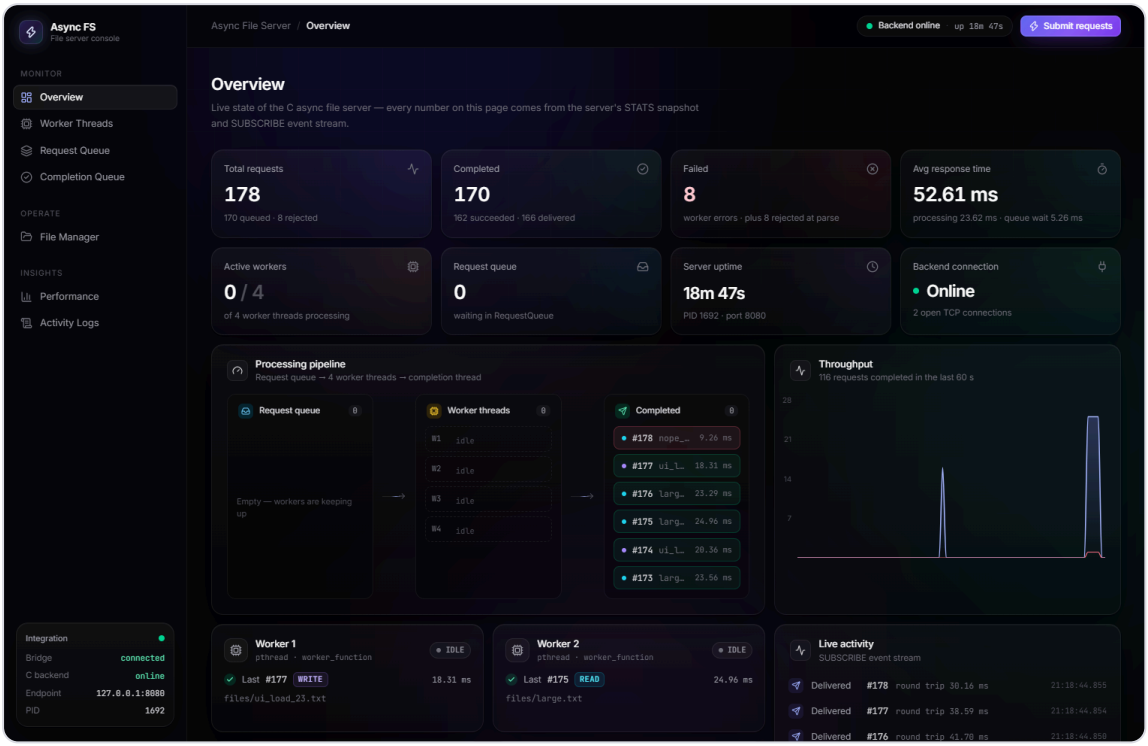
- Success is reported only after a `RESP … OK` from the server. Server error codes map to HTTP statuses: 404 (ENOENT), 409 (EEXIST), 400 (EINVAL), 413 (ETOOBIG).
- While the backend is offline, requests fail immediately with **503**. A connection lost mid-request returns **502** stating the outcome is unknown; a timeout returns **504**.
- Filenames are validated with the same rule as the C server, request bodies are capped at 12 MB, and the static file server cannot serve anything outside the dashboard build.

Keeping the dashboard consistent

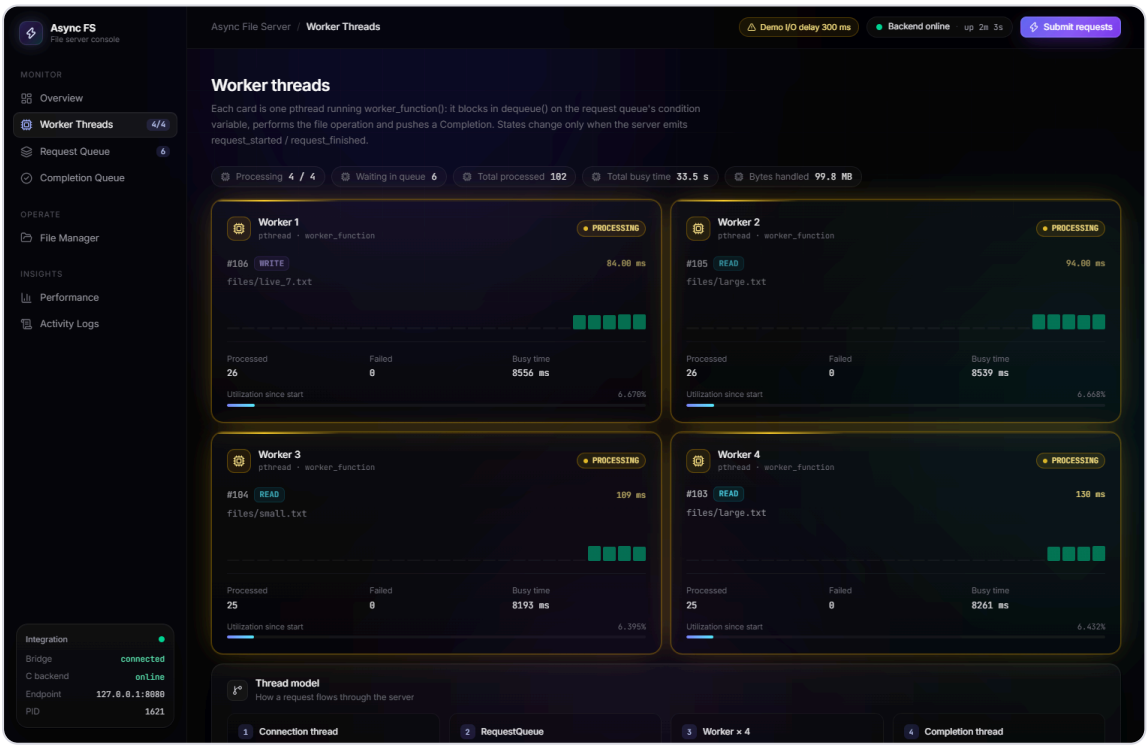
Every event carries a sequence number and every `STATS` snapshot records the latest sequence it includes. The browser applies events for immediate animation, and applies a snapshot only when it is at least as new as the last event seen. This prevents double counting and flicker. A change in the server's process ID or start time is detected as a restart and resets the view.

7 Dashboard

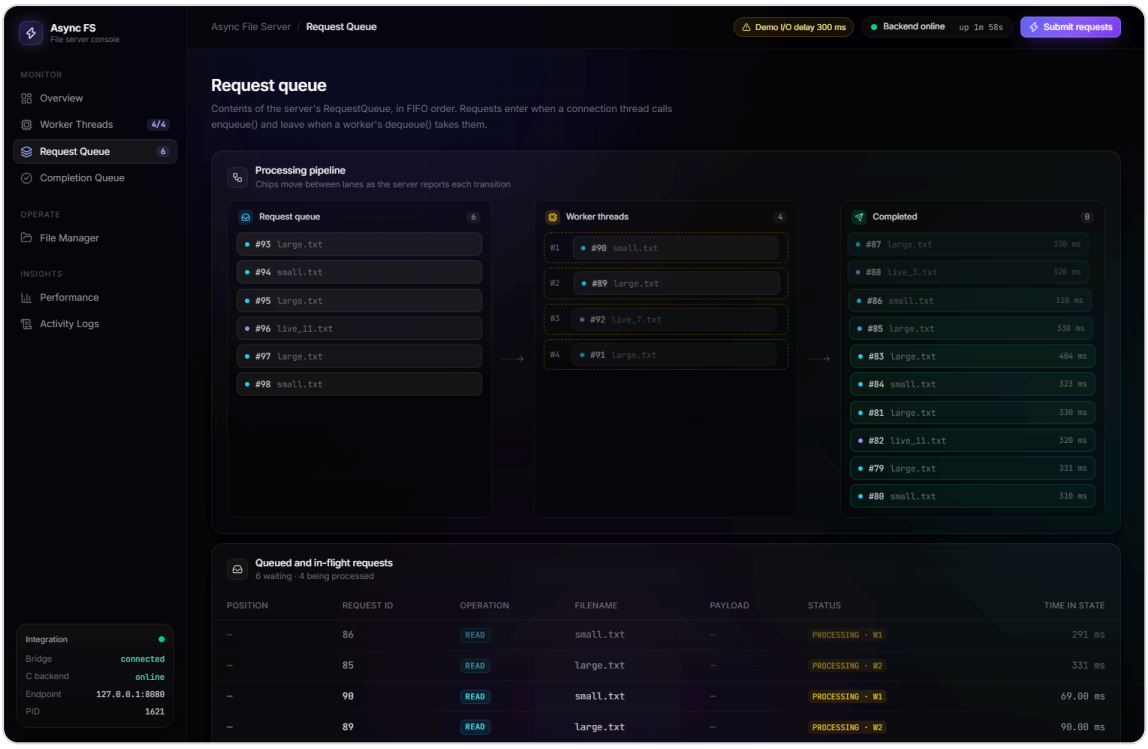
A dark, real-time console built with React, TypeScript, Tailwind CSS, Framer Motion, Lucide icons and Recharts. All animation is triggered by events from the server; nothing is simulated.



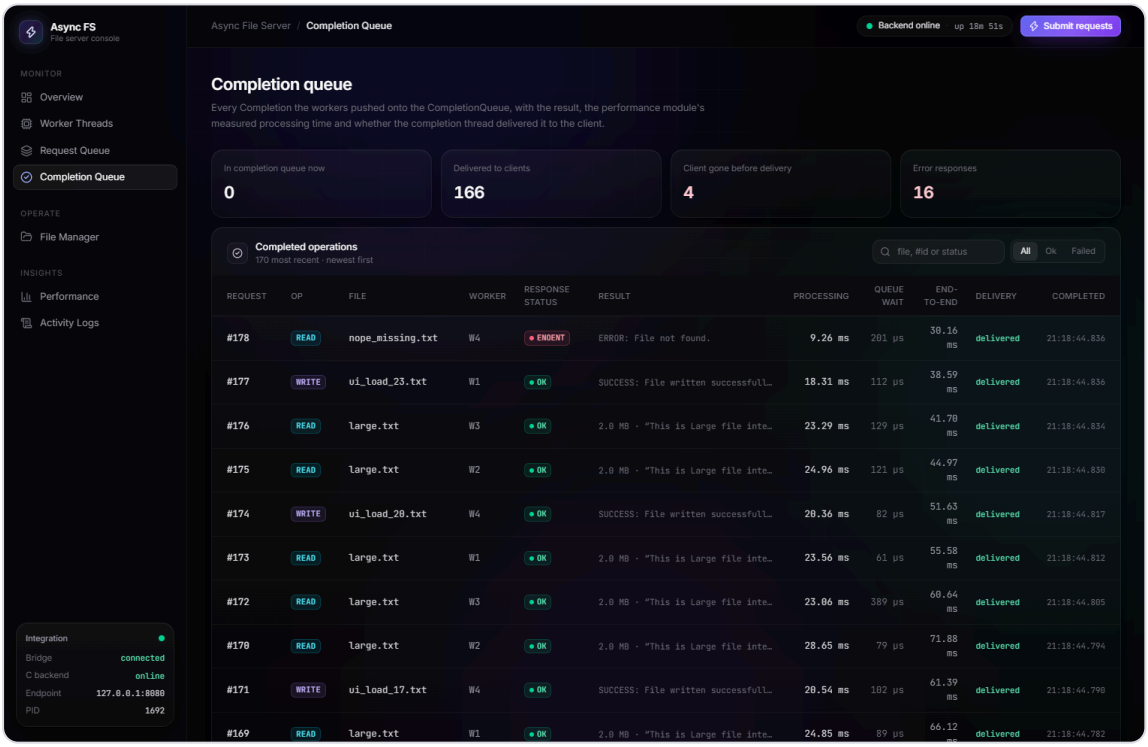
Overview. Total, completed and failed requests, average response time, active workers, queue size, uptime and connection status, with the live pipeline, throughput chart and activity stream.



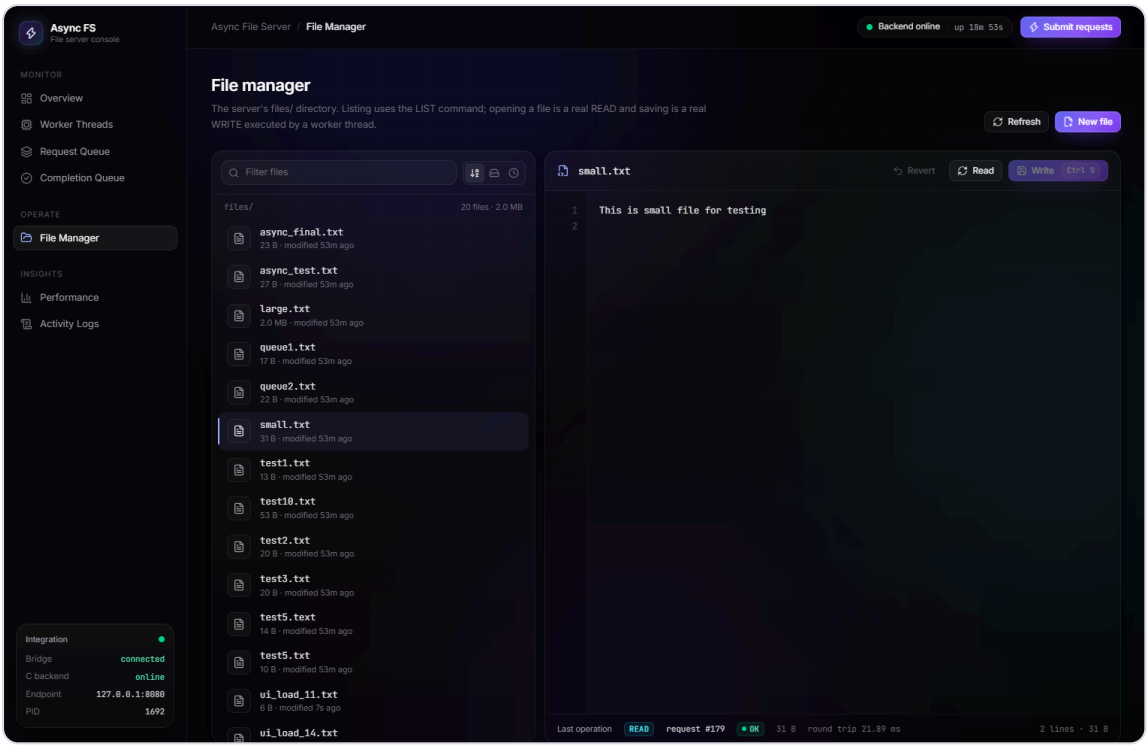
Worker threads under load. All four pthreads PROCESSING with their current request ID, operation, file and live elapsed time; six requests waiting. Captured with the opt-in `--io-delay-ms 300` demo flag, shown as a badge in the header.



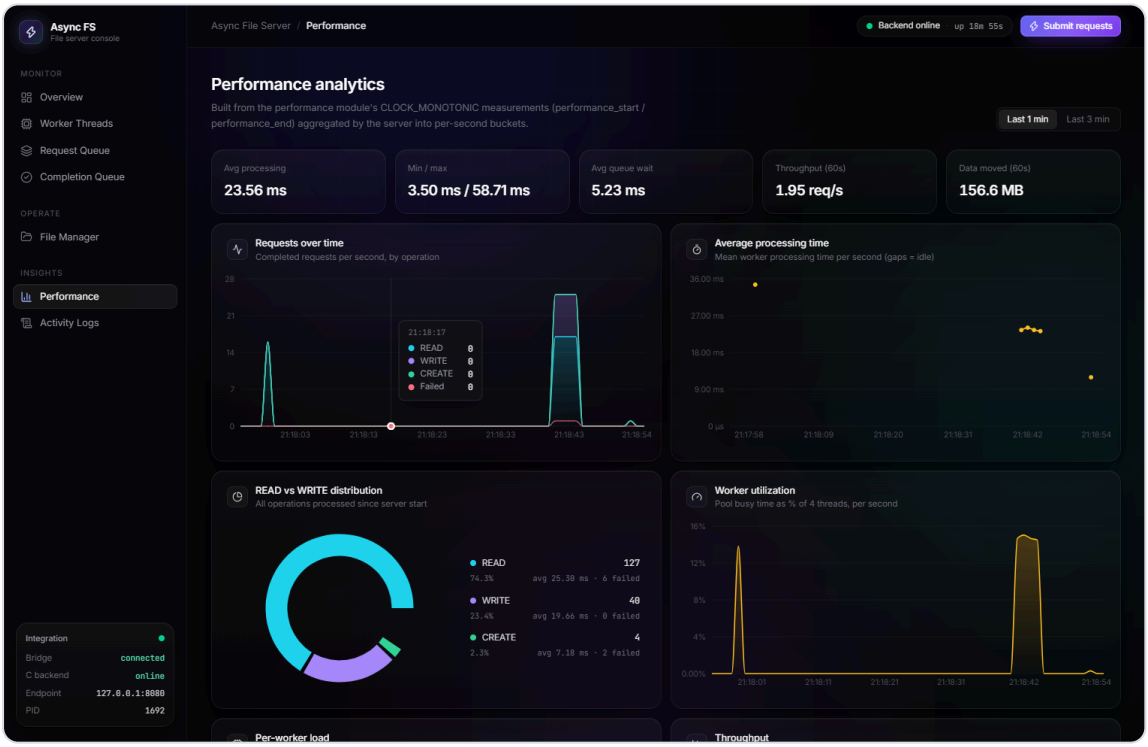
Request queue. The processing pipeline (queue → workers → completed) and a table of the real `RequestQueue` contents in FIFO order alongside in-flight requests.



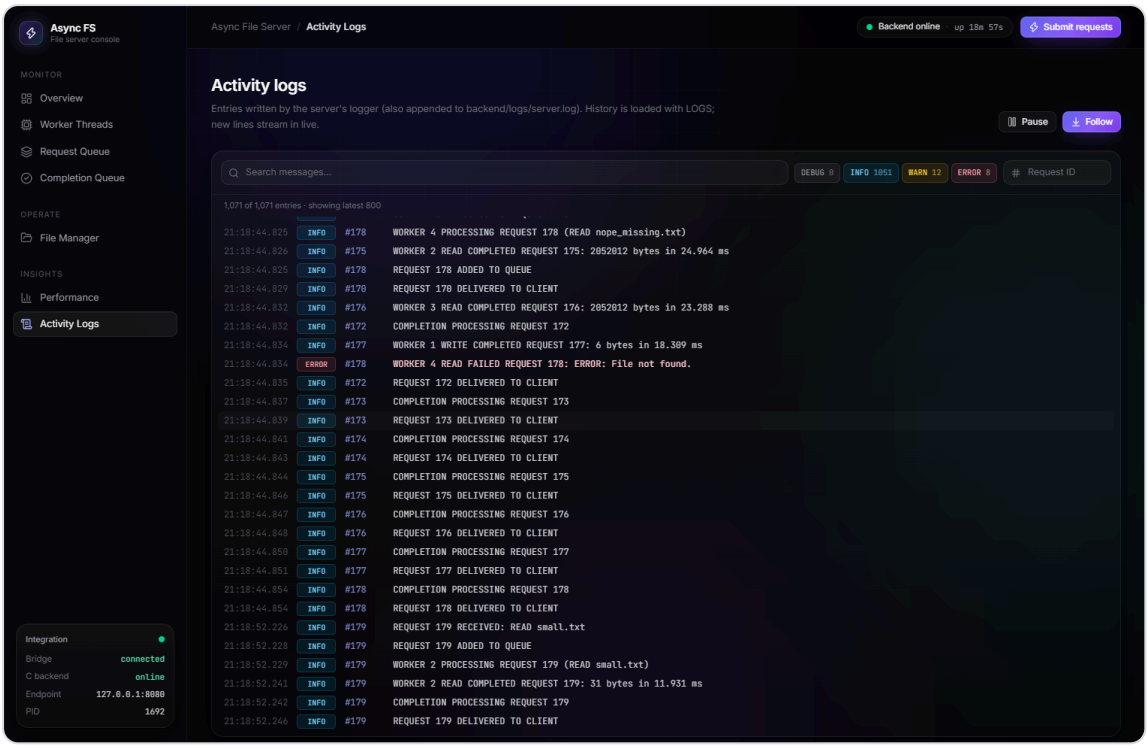
Completion queue. Every completion with response status, result, worker, measured processing time, queue wait, end-to-end latency and delivery outcome. The error row is a real `ENOENT` from the server.



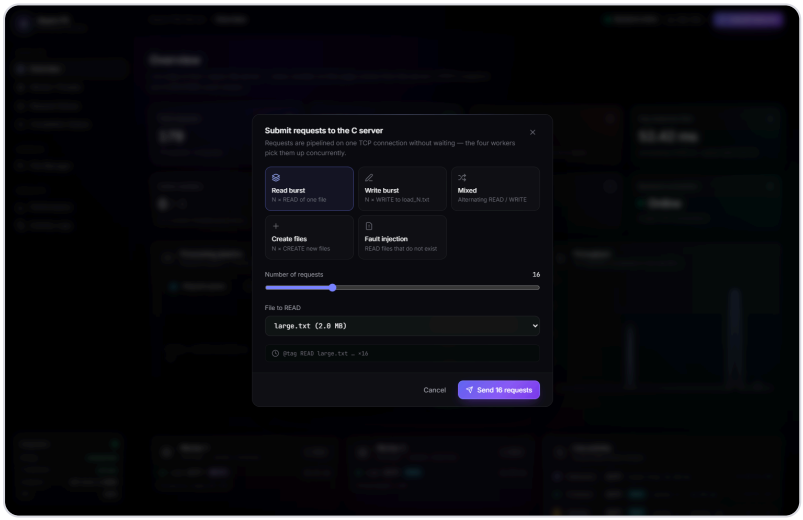
File manager. The server's `files/` directory via `LIST`. Opening a file is a real READ (request #102 shown in the status bar); saving with Ctrl+S is a real WRITE; new files use CREATE.



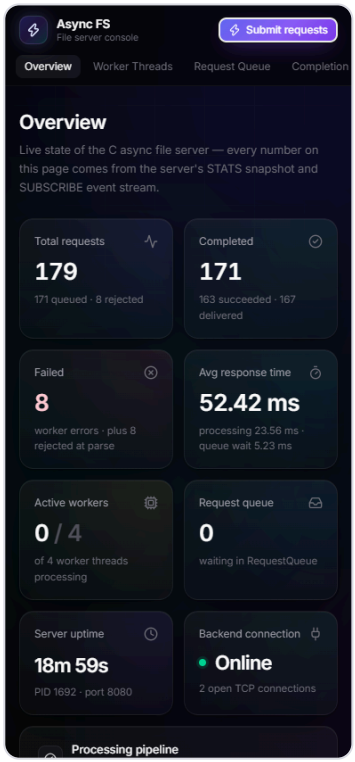
Performance analytics. Requests over time by operation, average processing time, READ/WRITE distribution and pool utilization, built from the performance module's per-request measurements. Further charts cover per-worker load, throughput, completion-time comparison and a per-request scatter.



Activity logs. Server log entries with timestamps, levels and request IDs, searchable and filterable by level and request; clicking an ID filters to that request's full lifecycle.



Submit requests. Sends real batches: read or write bursts, mixed, create, or fault injection (reads of missing files).



Responsive. Phone-width layout, no horizontal scrolling.

8 Testing and results

All suites were run against the live stack: C server in WSL Ubuntu (GCC 15.2), bridge on Node 24, dashboard in headless Chrome.

35/35	30/30	0	0
Raw TCP protocol checks	End-to-end bridge checks	ThreadSanitizer warnings	Browser console errors

REQUIREMENT	RESULT	EVIDENCE
Backend starts	Pass	Server listens on :8080; bridge reports it online within one second.
Frontend connects	Pass	WebSocket receives status, snapshot (4 workers) and live events.
Connection status is accurate	Pass	Status pushed on change; offline includes the reason.
Create a file	Pass	201 with a real request ID; a duplicate returns 409 EEXIST.
Write a file	Pass	Multi-line UTF-8 content written; byte count confirmed.
Read a file	Pass	Content returned byte-for-byte identical to what was written.
Multiple requests	Pass	16-request batch: 16 distinct IDs, all succeeded.
Four workers concurrently	Pass	All 4 workers used; up to 4 requests overlapping.
Completions reach the frontend	Pass	started / finished / delivered events for every batch request.
Metrics are real measurements	Pass	Every completion carries a measured processing time; timeline buckets match.
Logs update	Pass	History via LOGS and live log lines over WebSocket, tied to request IDs.
Backend shutdown → offline	Pass	Offline pushed; requests return 503; recovers automatically on restart.
Traversal and invalid input	Pass	`../`, `..`, hidden, absolute and slash-containing names rejected.
Disconnecting clients	Pass	Clients that vanish mid-request do not affect the server.
Original client compatibility	Pass	Unchanged `client_multi_async.c` works for single, batch and EXIT.

9 Installation and usage

Requirements

- WSL Ubuntu with `gcc` and `make` (`sudo apt install build-essential`)
- Node.js 18 or newer on Windows
- Python 3, only for the protocol test suite

One command (Windows PowerShell)

```
cd D:\OS_Project
powershell -ExecutionPolicy Bypass -File .\scripts\start-all.ps1     # opens
http://127.0.0.1:7070

.\scripts\start-all.ps1 -Dev             # hot-reloading dashboard on :5173
.\scripts\start-all.ps1 -IoDelayMs 300   # demo mode: visible queueing
.\scripts\stop-all.ps1                   # stop all; server prints its performance summary
```

Step by step

```
# 1. C backend (WSL)
wsl --cd /mnt/d/OS_Project/backend -- make
wsl --cd /mnt/d/OS_Project/backend -- ./server

# 2. Bridge
cd D:\OS_Project\bridge
npm install
node src/index.js

# 3. Dashboard (served by the bridge at http://127.0.0.1:7070)
cd D:\OS_Project\frontend
npm install
npm run build

# Original CLI client, alongside the dashboard
wsl --cd /mnt/d/OS_Project/backend -- ./client
```

Running the tests

```
python backend\tests\protocol_test.py
cd bridge
node test\bridge.test.js
node test\bridge.test.js --offline     # also stops and restarts the C server
```

10 Project structure

PATH	CONTENTS
`backend/src/server.c`	Accept loop, connection threads, workers, completion thread, protocol.
`backend/src/queue.[ch]`	Request and completion queues: original API plus observer hooks.
`backend/src/performance.[ch]`	CLOCK_MONOTONIC per-request timing and aggregate statistics.
`backend/src/logger.[ch]`	Log file, levels, request IDs, in-memory ring.
`backend/src/monitor.[ch]`	Worker state, completion history, timeline, event ring.
`backend/src/jsonbuf.[ch]`	JSON builder for monitoring output.
`backend/clients/client.c`	Original interactive multi-request client (unchanged).
`backend/legacy/`	Untouched copies of the original sources.
`backend/files/` · `backend/logs/`	Served directory and server log.
`backend/tests/protocol_test.py`	Raw TCP protocol test suite.
`bridge/src/`	`index.js` (HTTP + WebSocket), `backend.js` (TCP connections), `protocol.js` (frame parser).
`bridge/test/bridge.test.js`	End-to-end test suite.
`frontend/src/`	`lib/` (state store, API, types), `components/` (UI, layout, live views), `pages/` (seven pages).
`scripts/`	`start-all.ps1`, `stop-all.ps1`, `start-all.sh`.

A note on timing. Small file operations complete in microseconds, so the request queue normally drains before it can be seen. The `--io-delay-ms` flag makes each worker hold every request for a fixed time so queueing becomes visible in a demonstration. It is off by default, reported by the server in `STATS`, and displayed as a badge in the dashboard whenever it is active.